

NiceGUI 中 Tailwind CSS 的完整使用指南

Tailwind CSS 是一款实用优先的 CSS 框架，而 NiceGUI 作为轻量级的 Python Web UI 框架，深度集成了 Tailwind CSS，支持通过原生 Tailwind 语法、自定义类、覆盖默认样式等方式快速定制 UI 样式。以下从核心用法、进阶技巧、注意事项等维度全面阐述：

一、基础使用：直接应用 Tailwind 内置类

NiceGUI 所有组件都提供 `.classes()` 方法，可直接传入 Tailwind 内置类名，快速设置样式，这是最基础且常用的方式。

```
from nicegui import ui

# 示例：给按钮、标签添加 Tailwind 样式
ui.button("Primary Button").classes("bg-blue-600 hover:bg-blue-700 text-white px-4 py-2 rounded-lg")
ui.label("提示文本").classes("text-red-500 font-bold text-lg mt-4")

ui.run()
```

核心说明：

- `.classes()` 方法支持所有 Tailwind 核心类（布局、颜色、间距、状态等）；
- 多个类名用空格分隔，与原生 Tailwind 写法完全一致；
- 适用于所有 NiceGUI 组件（按钮、标签、行、列、输入框、卡片等）。

二、自定义组件类：@layer directive

当需要复用自定义样式时，可通过 `@layer components` 定义专属类，结合 `@apply` 复用 Tailwind 内置类，实现样式封装。

1. 核心语法

通过 `ui.add_head_html()` 注入 `<style type="text/tailwindcss">` 标签，在 `@layer components` 中定义自定义类：

```
from nicegui import ui

# 注入自定义 Tailwind 组件类
ui.add_head_html('''
    <style type="text/tailwindcss">
        @layer components {
            /* 自定义可复用类：蓝色卡片 */
            .custom-blue-card {
                @apply bg-blue-500 text-white p-6 rounded-xl shadow-lg hover:bg-blue-600
                transition-colors;
            }
            /* 自定义可复用类：灰色按钮 */
            .custom-gray-btn {
                @apply bg-gray-200 text-gray-800 px-4 py-2 rounded-md hover:bg-gray-300;
            }
        }
    </style>
''')
```

```

    }
</style>
'''

# 应用自定义类
with ui.column():
    ui.label("自定义蓝色卡片").classes("custom-blue-card")
    ui.button("灰色按钮").classes("custom-gray-btn mt-4")

ui.run()

```

2. 关键注意事项

- 样式标签必须指定 `type="text/tailwindcss"`，否则 NiceGUI 无法识别 Tailwind 语法，样式会失效；
- `@layer components` 是 Tailwind 规范，确保自定义类优先级与内置组件类一致，避免样式冲突；
- `@apply` 后可拼接任意 Tailwind 内置类，支持状态类（`hover:`、`active:`）、过渡类（`transition-*`）等；
- 自定义类名遵循 CSS 命名规则（小写、横线分隔），可在任意组件的 `.classes()` 中复用。

三、覆盖 Tailwind 默认样式

Tailwind 会重置 HTML 原生元素的默认样式（如 `h1-h6` 字体大小、`body` 行高、按钮默认样式等），若需恢复 / 修改这些默认样式，可直接在 `text/tailwindcss` 样式标签中覆盖原生元素样式。

1. 基础示例：修改原生标签样式

```

from nicegui import ui

# 覆盖 h2 标签的默认样式
ui.add_head_html('''
<style type="text/tailwindcss">
    /* 覆盖 h2 原生样式（Tailwind 默认重置了 h2 字体大小） */
    h2 {
        @apply text-3xl font-bold text-green-700 mb-4;
    }
    /* 覆盖 body 全局样式 */
    body {
        @apply bg-gray-50;
    }
</style>
''')

# 渲染原生 h2 标签（sanitize=False 允许原生 HTML）
ui.html('<h2>自定义 H2 标题</h2>', sanitize=False)

# 普通组件也会继承 body 全局样式
ui.label("页面全局背景为浅灰色").classes("text-lg")

ui.run()

```

2. 关键说明

- 无需包裹在 `@layer` 中，直接针对 HTML 原生元素（`h1`、`body`、`button` 等）写样式；
- 仍可使用 `@apply` 复用 Tailwind 类，也可混合原生 CSS 属性（如 `font-size: 24px;`）；
- 必须使用 `type="text/tailwindcss"`，否则自定义样式会被 Tailwind 的默认重置样式覆盖；
- 适用于全局样式调整（如页面背景、默认字体、标题样式）。

四、高级技巧

1. 结合条件动态切换类

NiceGUI 支持动态修改组件类，结合 Tailwind 实现状态切换：

```
from nicegui import ui

# 初始状态：红色文本
label = ui.label("动态样式示例").classes("text-red-500 font-bold")

# 按钮切换样式（绿色/红色）
def toggle_style():
    if "text-red-500" in label.classes:
        label.classes(remove="text-red-500", add="text-green-500")
    else:
        label.classes(remove="text-green-500", add="text-red-500")

ui.button("切换颜色", on_click=toggle_style).classes("mt-4")

ui.run()
```

2. 响应式样式

直接使用 Tailwind 响应式前缀（`sm:`、`md:`、`lg:` 等），适配不同屏幕尺寸：

```
from nicegui import ui

# 响应式布局：小屏堆叠，大屏并排
with ui.row().classes("flex flex-col sm:flex-row gap-4"):
    ui.card().classes("bg-blue-100 p-4 sm:w-1/2 lg:w-1/3")
    ui.card().classes("bg-green-100 p-4 sm:w-1/2 lg:w-1/3")

ui.run()
```

3. 自定义 Tailwind 配置（扩展主题）

若需扩展 Tailwind 主题（如自定义颜色、字体、间距），可通过注入 `tailwind.config` 实现：

```
from nicegui import ui

# 注入自定义 Tailwind 配置 + 自定义类
ui.add_head_html('''
    <script>
        tailwind.config = {
```

```

        theme: {
            extend: {
                colors: {
                    // 自定义颜色：主色调
                    primary: '#165DFF',
                },
                spacing: {
                    // 自定义间距
                    '18': '4.5rem',
                }
            }
        }
    }
</script>
<style type="text/tailwindcss">
    @layer components {
        .custom-primary-btn {
            @apply bg-primary text-white px-6 py-2 rounded-lg;
        }
    }
</style>
'''

ui.button("自定义主色调按钮").classes("custom-primary-btn mt-4")
ui.label("自定义间距").classes("mt-18")

ui.run()

```

五、常见问题与注意事项

1. 样式失效排查：

- 确认样式标签 `type="text/tailwindcss"` 未遗漏；
- 自定义类名无拼写错误，`@apply` 后类名与 Tailwind 内置类一致；
- 避免使用 Tailwind 未内置的类（需通过主题扩展或原生 CSS 实现）。

2. 优先级问题：

- `@layer components` 定义的类优先级低于行内样式（`.style()` 方法），高于全局重置样式；
- 如需强制覆盖，可使用 `!important`（如 `@apply bg-red-500 !important;`）。

3. 性能优化：

- 避免定义过多未使用的自定义类，Tailwind 会按需生成样式，但冗余类仍会增加 HTML 体积；
- 全局样式尽量集中注入，避免多次调用 `ui.add_head_html()`。

NiceGUI 中 Tailwind CSS 的全面使用指南

Tailwind CSS 是 NiceGUI 的核心样式框架（默认内置），提供原子化 CSS 类名实现高效样式定制。NiceGUI 不仅原生支持 Tailwind 基础用法，还适配了其 `@layer` 指令、`@apply` 语法，并通过 `ui.add_head_html()`/`ui.add_css()` 等方式扩展自定义 Tailwind 类，是 NiceGUI 样式开发的首选方式。

一、Tailwind CSS 核心基础 (NiceGUI 原生支持)

1. 直接使用内置 Tailwind 类

NiceGUI 所有元素均可通过 `classes()` 方法直接绑定 Tailwind 原子类，无需额外配置，覆盖布局、颜色、尺寸、间距、阴影等所有场景。

基础示例：

```
from nicegui import ui

# 按钮：红色背景、白色文字、圆角、内边距、hover 效果
ui.button("Tailwind Button")\
    .classes("bg-red-500 text-white rounded-lg p-4 hover:bg-red-600 transition-colors")

# 卡片：宽度、阴影、圆角、内边距、响应式宽度
ui.card()\
    .classes("w-80 sm:w-96 lg:w-[640px] shadow-lg rounded-xl p-6 bg-white")\
    .add(ui.label("Tailwind Card").classes("text-xl font-bold mb-2 text-gray-800"))

# 布局：Flex 容器（水平居中、垂直对齐、间距）
ui.row().classes("flex justify-center items-center gap-4 p-8")\
    .add(ui.input("用户名").classes("w-64 border-gray-300 focus:border-blue-500 rounded-md"))\
    .add(ui.button("提交").classes("bg-blue-500 text-white px-4 py-2 rounded-md"))

ui.run()
```

2. Tailwind 核心特性支持

特性	示例用法	说明
响应式前缀	<code>sm:w-64 md:w-96 lg:w-[640px]</code>	适配不同屏幕尺寸
伪类 / 伪元素	<code>hover:bg-red-600 focus:outline-none</code>	交互态样式
重要性 (!important)	<code>!bg-red-500</code>	强制覆盖样式
数值化类名	<code>w-[300px] h-[200px] gap-[16px]</code>	自定义数值（需用中括号）
颜色系统	<code>bg-blue-500 text-gray-800 border-green-300</code>	内置 100-900 色阶
间距 / 尺寸	<code>p-4 (padding: 1rem) m-2 (margin: 0.5rem)</code>	基于 rem 的间距体系

二、自定义 Tailwind 类 (@layer + @apply)

NiceGUI 支持通过 `ui.add_head_html` 定义 Tailwind 自定义类（需用 `type="text/tailwindcss"` 的 `<style>` 标签），结合 Tailwind 的 `@layer` 指令将自定义类归入指定层，实现样式复用。

1. 核心语法规则

- 必须使用 `<style type="text/tailwindcss">` (而非 `text/css`) , 否则 Tailwind 语法 (如 `@apply`) 不生效;
- 推荐将自定义类归入 `components` 层 (NiceGUI 适配的 Tailwind 层) , 便于优先级管理;
- `@apply` 可批量复用 Tailwind 原子类, 简化自定义类定义。

2. 基础示例 (自定义组件类)

```
from nicegui import ui

# 定义 Tailwind 自定义类 (components 层)
ui.add_head_html('''
    <style type="text/tailwindcss">
        @layer components {
            /* 自定义蓝色盒子类 */
            .blue-box {
                @apply bg-blue-500 p-12 text-center shadow-lg rounded-lg text-white;
            }
            /* 自定义按钮类 */
            .btn-primary {
                @apply bg-indigo-600 text-white px-6 py-3 rounded-md hover:bg-indigo-700
                focus:ring-2 focus:ring-indigo-500 focus:outline-none transition-all;
            }
            /* 自定义输入框类 */
            .input-default {
                @apply w-72 border-gray-300 rounded-md px-4 py-2 focus:border-indigo-500
                focus:ring-1 focus:ring-indigo-500;
            }
        }
    </style>
''')

# 复用自定义类
with ui.row().classes("flex gap-6 justify-center p-8"):
    ui.label("Hello").classes("blue-box")
    ui.label("World").classes("blue-box")

ui.input("用户名").classes("input-default")
ui.button("提交").classes("btn-primary")

ui.run()
```

3. 自定义工具类 (utilities 层)

如需扩展通用工具类 (如自定义间距、颜色) , 可归入 `utilities` 层:

```
ui.add_head_html('''
    <style type="text/tailwindcss">
        @layer utilities {
            /* 自定义间距工具类 */
            .px-10 {
```

```

        @apply px-[2.5rem];
    }
    /* 自定义文字渐变工具类 */
    .text-gradient {
        @apply bg-clip-text text-transparent bg-gradient-to-r from-purple-500 to-
pink-500;
    }
    /* 自定义旋转动画 */
    .rotate-hover {
        @apply transition-transform hover:rotate-3;
    }
}
</style>
'''

ui.label("渐变文字+旋转").classes("text-xl font-bold text-gradient rotate-hover")
ui.card().classes("px-10 py-6 bg-white shadow-md")

```

三、高级用法

1. 结合响应式与自定义类

```

ui.add_head_html(''
    <style type="text/tailwindcss">
        @layer components {
            .responsive-card {
                @apply w-64 sm:w-80 md:w-96 lg:w-[500px] rounded-xl shadow-md p-6 bg-white;
            }
        }
    </style>
'')

ui.card().classes("responsive-card")\
    .add(ui.label("响应式卡片").classes("text-lg font-bold"))

```

2. 自定义 Tailwind 主题 (扩展颜色 / 字体)

NiceGUI 支持通过 `tailwind.config` 扩展 Tailwind 原生主题，需在 `ui.add_head_html` 中定义配置：

```

ui.add_head_html(''
    <!-- 配置 Tailwind 主题 -->
    <script>
        tailwind.config = {
            theme: {
                extend: {
                    colors: {
                        // 自定义品牌色
                        brand: {
                            50: '#f0f9ff',
                            500: '#0ea5e9',
                            700: '#0369a1',

```

```

        },
    },
    fontFamily: {
        // 自定义字体
        sans: ['Inter', 'system-ui', 'sans-serif'],
    },
}

}

}

</script>
<!-- 使用自定义主题类 -->
<style type="text/tailwindcss">
    @layer components {
        .brand-btn {
            @apply bg-brand-500 text-white px-6 py-2 rounded-md hover:bg-brand-700;
        }
    }
</style>
'''

# 应用自定义主题类
ui.button("品牌按钮").classes("brand-btn")
ui.label("自定义字体").classes("font-sans text-brand-500 text-xl")

```

3. 动态切换 Tailwind 类

结合 NiceGUI 的响应式变量，动态修改元素的 Tailwind 类：

```

from nicegui import ui

# 响应式变量控制样式
is_dark = ui.reactive(False)

def toggle_theme():
    is_dark.value = not is_dark.value
    # 动态更新类名
    card.classes(
        add="bg-gray-800 text-white" if is_dark.value else "",
        remove="bg-white text-gray-800" if is_dark.value else ""
    )

# 初始卡片样式
card = ui.card().classes("w-64 p-6 bg-white text-gray-800 rounded-lg shadow-md transition-colors")
card.add(ui.label("动态主题卡片"))

ui.button("切换主题", on_click=toggle_theme).classes("mt-4 bg-blue-500 text-white px-4 py-2 rounded-md")

ui.run()

```


4. 结合 Tailwind 插件 (如 @tailwindcss/forms)

NiceGUI 支持引入 Tailwind 官方插件, 需先通过 CDN 加载插件, 再配置使用:

```
ui.add_head_html('''
    <!-- 加载 Tailwind forms 插件 -->
    <script src="https://cdn.tailwindcss.com/plugins/forms.js"></script>
    <!-- 配置插件并定义样式 -->
    <script>
        tailwind.config = {
            plugins: [tailwindcssForms],
        }
    </script>
    <style type="text/tailwindcss">
        @layer components {
            .form-input {
                @apply w-72 rounded-md border-gray-300 focus:border-blue-500 focus:ring-
blue-500;
            }
        }
    </style>
''')

# 应用 forms 插件样式的输入框
ui.input("带表单样式的输入框").classes("form-input")
ui.select(["选项1", "选项2"]).classes("form-input")
```

四、关键注意事项

- 样式标签类型:** 定义 Tailwind 自定义类时, `<style>` 标签必须设为 `type="text/tailwindcss"`, 否则 `@apply`/`@layer` 语法不生效;
- 层优先级:** NiceGUI 中 Tailwind 的 `components` 层优先级低于 `utilities` 层, 自定义类建议优先放入 `components`, 工具类放入 `utilities`;
- 与原生 CSS 混用:** 可在 `type="text/tailwindcss"` 标签中混合原生 CSS, 但 Tailwind 语法需通过 `@apply` 调用;
- 性能优化:** Tailwind 原子类不会产生冗余 CSS (NiceGUI 内置 PurgeCSS 自动清理未使用的类), 无需担心样式体积;
- 类名冲突:** 自定义类名避免与 Tailwind 内置类名重复 (如 `btn` 已被 Quasar 占用, 建议加前缀如 `my-btn`);
- 调试技巧:** 在浏览器开发者工具的「Elements」面板查看元素的 `class` 属性, 「Styles」面板验证 Tailwind 样式是否生效。